56

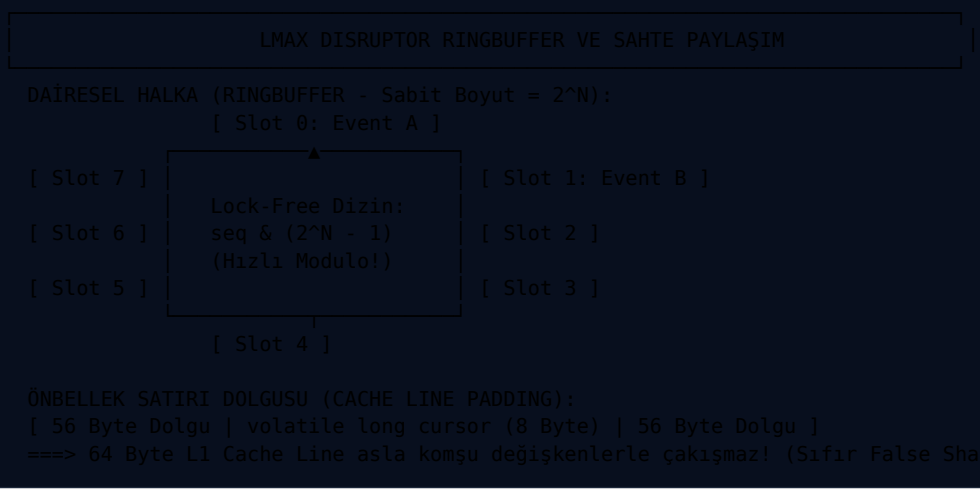
BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

MAANG Vaka 2: LMAX Disruptor Deseni & Lock-Free RingBuffer ile Ultra-Düşük Gecikme

MAANG Case 2: LMAX Disruptor & Lock-Free RingBuffer with Cache Padding

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Disruptor, Londra borsasındaki saniyede 6 milyon emri sıfır gecikmeyle işlemek için icat edilmiştir. Klasik kuyruklar iki kişinin birbirinin eline mektup tutuşturması gibidir (biri verirken diğeri bekler). RingBuffer ise döner bir dönme dolaptır. Sepetler önceden hazırdır ve hiç durmadan döner. Üretici sıradaki boş sepete kâğıdı fırlatır, tüketici arkasından gelip alır; ne kilit vardır ne bekleme! Bellek önbelleklerini (CPU Cache) korumak için de değişkenlerin sağına soluna boş dolgu konur, böylece çekirdekler birbirini hiç rahatsız etmez.

🇬🇧 ENGLISH (ELI5)

The LMAX Disruptor was engineered to process 6,000,000 orders/sec with nanosecond latency in financial exchanges. Standard queues use locks and dynamically allocate nodes, hammering the garbage collector. The Disruptor pre-allocates an immutable circular carousel (`RingBuffer`). Producers claim slots using monotonic atomic sequences, overwrite pre-allocated event slots in-place, and consumers read behind them with zero locks. Cache-line padding ensures CPU L1/L2 caches never suffer false sharing.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LMAX Disruptor'ın 4 Temel Mimari Sırrı: 1) Sıfır Çöp Üretimi (Zero-Allocation): RingBuffer boyutu 2'nin kuvvetidir (2^N). Tüm olay nesneleri uygulama başlarken heap'te önceden tahsis edilir (`pre-allocation`). Çalışma anında asla `new` çağrılmaz; aynı bellek yuvaları ezilerek yeniden kullanılır, GC duraklamaları (Stop-the-World) tamamen yok edilir. 2) İkili Maskeleme (Bitwise Indexing): Dizi indeksi hesaplanırken yavaş olan modulo (`%`) yerine `sequence & (bufferSize - 1)` bit düzeyinde VE işlemi kullanılır; tek bir CPU çevriminde slot bulunur. 3) Kilit-Siz Sıralama (Single-Writer Principle): Üretici sırayı tekil `volatile long cursor` ile takip eder; okuyucular CAS döngüsü olmadan `VarHandle.getAcquire` veya `Volatile` ile takip eder. 4) Sahte Paylaşım Önleme: Sıra değişkenleri arasına 56 baytlık dolgu (`padding`) yerleştirilerek değişkenin tek başına 64 baytlık L1 CPU Cache Line'ı kaplaması sağlanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Mechanical Sympathy of the LMAX Disruptor: 1) Zero Garbage Allocation: Array capacity is constrained to powers of two (2^N). Event slots are pre-allocated during initialization. Mutating state involves writing in-place to pre-existing slots, eliminating memory churn and GC pauses. 2) Bitwise Power-of-Two Indexing: Modulo arithmetic is replaced by bitwise masking (`sequence & (bufferSize - 1)`), executing in a single CPU clock cycle. 3) Lock-Free Mechanical Sympathy: Discards blocking queues and monitor locks in favor of monotonic sequence barriers backed by memory-fenced primitives. 4) Cache-Line Isolation: Sequence counters are isolated on separate 64-byte L1 cache lines using padding fields or `\@jdk.internal.vm.annotation.Contended`, preventing MESI cache invalidation storms.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Basitleştirilmiş Lock-Free RingBuffer Prensibi
import java.util.concurrent.atomic.AtomicLong;

public class MinimalDisruptorRingBuffer<T> {
    private final Object[] ringBuffer;
    private final int mask;
    // Cache Line Padding simülasyonu
    private final AtomicLong cursor = new AtomicLong(-1);

    public MinimalDisruptorRingBuffer(int powerOfTwoSize) {
        if (Integer.bitCount(powerOfTwoSize) != 1) {
            throw new IllegalArgumentException("Boyut 2'nin kuvveti olmalıdır!");
        }
        this.ringBuffer = new Object[powerOfTwoSize];
        this.mask = powerOfTwoSize - 1;
    }

    public long nextWriteSequence() {
        return cursor.incrementAndGet();
    }

    public void publish(long seq, T event) {
        int index = (int) (seq & mask); // Bitwise modulo (Işık hızında!)
        ringBuffer[index] = event;
    }

    @SuppressWarnings("unchecked")
    public T read(long seq) {
        int index = (int) (seq & mask);
        return (T) ringBuffer[index];
    }

    void main() {
        var buffer = new MinimalDisruptorRingBuffer<String>(1024);
        long seq = buffer.nextWriteSequence();
        buffer.publish(seq, "BIST100_LIMIT_ORDER_BUY_1000");

        String order = buffer.read(seq);
        System.out.println("RingBuffer'dan Okunan Emir: " + order);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

RingBuffer boyutu kesinlikle 2'nin kuvveti (1024, 4096, 65536) olmalıdır! Aksi halde `seq & (size - 1)` formülü çalışmaz ve dizi sınırları dışına taşma (ArrayIndexOutOfBoundsException) oluşur.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

RingBuffer capacity MUST strictly be a power of two (2^N). If breached, the bitwise indexing equation `sequence & (capacity - 1)` produces illegal indices, causing runtime exceptions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

🎯 Key Takeaways for the Interview

• Disruptor sıfır çöp tahsisi, 2'nin kuvveti bit maskelemesi ve cache-line padding ile kilit almadan mikrosaniye altı gecikme üretir.

• The Disruptor achieves sub-microsecond latency through zero GC allocation, bitwise power-of-two ring indexing, and false-sharing cache padding.